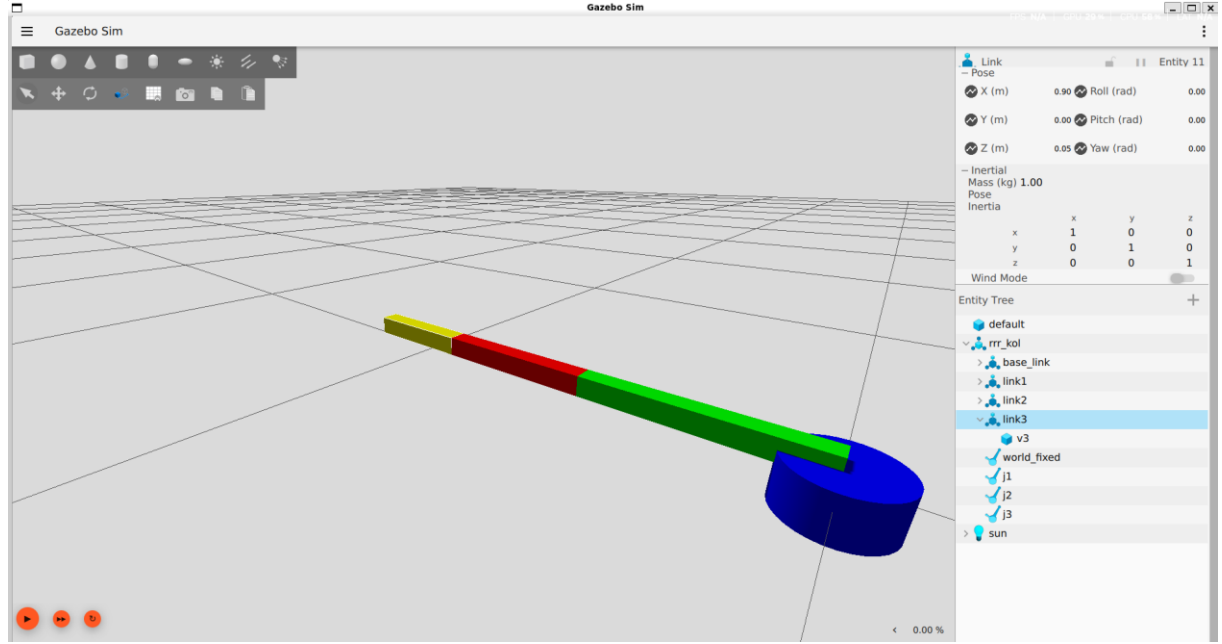




3-Serbestlik Dereceli RRR Planar Robot Manipulatörünün Kinematik Analizi ve Simülasyonu

ÖZET Bu raporda, üç serbestlik derecesine sahip (3-DoF) RRR (Revolute-Revolute-Revolute) planar bir robot manipulatörünün ileri ve ters kinematik analizleri gerçekleştirilmiş, çalışma alanı (workspace) algoritmik olarak türetilmiş ve Gazebo ortamında fiziksel simülasyonu yapılmıştır. Sistemin sadece X-Y düzleminde hareket edebilme kısıtı matematiksel olarak ispatlanmış ve hedef koordinatlardaki fiziksel imkansızlıklar analitik yöntemlerle tespit edilmiştir. Elde edilen teorik bulgular, simülasyon ortamından alınan telemetri verileriyle karşılaştırılarak hata analizi ile doğrulanmıştır.

Anahtar Kelimeler: İleri Kinematik, Ters Kinematik, RRR Planar Robot, Gazebo Simülasyonu, Çalışma Alanı.

github.com/damlablglc/3DoF-RRR-Planar-Robot

1. GİRİŞ (INTRODUCTION)

Robotik manipulatörlerin endüstriyel uygulamalarda doğru, verimli ve güvenli bir şekilde görev yapabilmesi için kinematik modellerinin eksiksiz bir şekilde çıkarılması gerekmektedir [1]. Kinematik analiz, robotun eklem açıları ile uç işlevcisinin (end-effector) uzaydaki konumu arasındaki matematiksel ilişkiyi tanımlar. Bu çalışmada incelenen 3-DoF RRR Planar robot, birbirine paralel **üç adet döner (revolute) mafsaldan** oluşmakta olup, **hareket kabiliyeti yalnızca iki boyutlu uzayla (X-Y düzlemi) sınırlıdır**. Bu projenin temel amacı, belirlenen uzuv uzunluklarına ($L_1=0.5m$ $L_2=0.4m$ $L_3=0.3m$) sahip robot kolunun ileri ve ters kinematik denklemlerini oluşturmak, çalışma alanını haritalamak ve Gazebo simülatörü kullanılarak teorik model ile fiziksel simülasyon arasındaki tutarlılığı hata analizi ile doğrulamaktır.

2. YÖNTEM (METHODS)

Projede robotun kinematik modellemesi Python programlama dili ve NumPy kütüphanesi kullanılarak analitik olarak gerçekleştirilmiştir. Görselleştirme aşamaları Matplotlib ile, fiziksel simülasyon ise Gazebo Harmonic motoru kullanılarak SDF (Simulation Description Format) yapısında oluşturulmuştur.

2.1. İleri Kinematik (Forward Kinematics) İleri kinematik problemi, verilen eklem açılarına (q_1, q_2, q_3) göre uç noktanın uzaydaki (P_x, P_y) koordinatlarının bulunması işlemidir. Trigonometrik izdüşümler kullanılarak sistemin ileri kinematik denklemleri aşağıdaki gibi tanımlanmıştır:

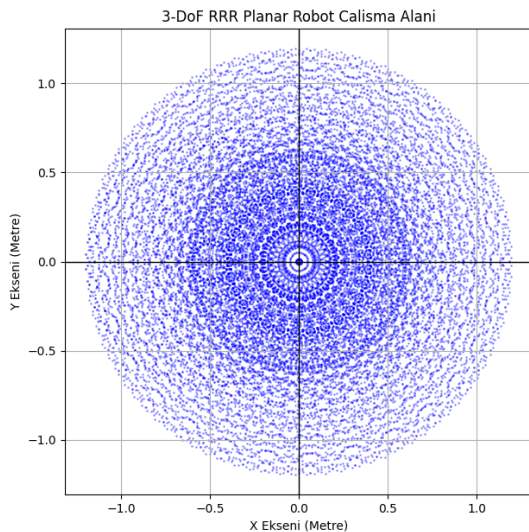
$$p_x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3)$$
$$p_y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) + L_3 \sin(q_1 + q_2 + q_3)$$

2.2. Ters Kinematik (Inverse Kinematics) Hedeflenen uç nokta pozisyonuna ulaşmak için gerekli eklem açılarının bulunması Kosinüs teoremi ve geometrik yaklaşımlarla çözülmüştür. Sistemin planar yapısı gereği, uzaydaki Z eksen konumu (P_z) robot mekaniği için kısıtlayıcı bir faktördür.

2.3. Hata Analizi Hesaplaması Simülasyondan elde edilen pose değerleri ile Python üzerinden hesaplanan teorik değerler arasındaki sapma, Öklid mesafesi formülü kullanılarak hesaplanmıştır.

3. BULGULAR (RESULTS)

Python algoritması kullanılarak $\pm\pi$ radyan sınırları içinde tüm olası eklem açısı kombinasyonları taranmış ve robotun ulaşabileceği noktalar kümesi hesaplanmıştır. Elde edilen çalışma alanı Şekil 1'de sunulmuştur.



Şekil 1: 3-DoF RRR Planar Robot Kolunun Çalışma Alanı (Workspace) Dağılımı

3.1. Ters Kinematik Analiz Bulguları Sistemin ters kinematik doğrulamasında hedef koordinat olarak $(X, Y, Z) = (0.8, 0.2, 0.1)$ noktası sisteme girdi olarak verilmiştir. Algoritma, hedef noktanın $Z = 0.1$ metre şartını fiziksel bir ihlal olarak algılamış ve "Çözüm Yok" çıktısı vererek hesaplamayı reddetmiştir.

```
kinematik.py > ...
34 def ters_kinematik(hedef_x, hedef_y, hedef_z, hedef_phi):
52
53     q2 = np.arccos(cos_q2)
54
55     k1 = L1 + L2 * np.cos(q2)
56     k2 = L2 * np.sin(q2)
57     q1 = np.arctan2(pwy, pwx) - np.arctan2(k2, k1)
58     q3 = hedef_phi - q1 - q2
59
60     print("BAŞARILI: Çözüm Bulundu (Dirsek Aşağı):")
61     print(f"q1 = {np.degrees(q1):.2f} derece")
62     print(f"q2 = {np.degrees(q2):.2f} derece")
--

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

--- Örnek Hesaplama Sonucu (İleri Kinematik) ---
X Pozisyonu: 0.571 metre
Y Pozisyonu: 0.495 metre

Çalışma alanı (Workspace) hesaplanıyor, lütfen bekleyin...

--- Ters Kinematik (IK) Hedefi: (X:0.8, Y:0.2, Z:0.1) ---
HATA: Çözüm Yok!
Neden: RRR Planar kol sadece XY düzleminde (Z=0) hareket edebilir.
Z eksenini 0 olmadığı için bu noktaya fiziksel olarak ulaşamaz.

--- Ters Kinematik (IK) Hedefi: (X:0.5, Y:0.5, Z:0.0) ---
BAŞARILI: Çözüm Bulundu (Dirsek Aşağı):
q1 = -6.08 derece
q2 = 127.64 derece
q3 = -76.55 derece

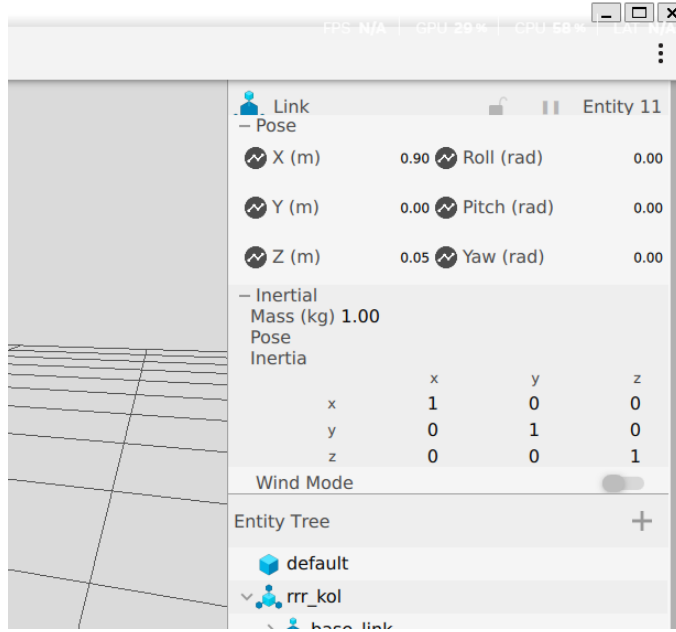
PS C:\Users\daml\l\Desktop\robotik_odev1\robotikodev1>
```

Şekil 2: Hatalı Z-Eksenini Hedefi İçin Ters Kinematik Algoritması Çıktısı

3.2. Simülasyon ve Hata Analizi Bulguları

Robotun tüm eklem açıları $q_1=0$, $q_2=0$, $q_3=0$ radyan konumundayken Gazebo simülatöründen alınan telemetri verileri ile teorik model karşılaştırılmıştır.

Tablo 1: Teorik Model ile Gazebo Simülasyonu Konum Değerlerinin Karşılaştırması



Link		Entity 11	
- Pose			
X (m)	0.90	Roll (rad)	0.00
Y (m)	0.00	Pitch (rad)	0.00
Z (m)	0.05	Yaw (rad)	0.00
- Inertial			
Mass (kg) 1.00			
Pose			
Inertia			
	x	y	z
x	1	0	0
y	0	1	0
z	0	0	1
Wind Mode			
Entity Tree			
default			
rrr_kol			
base_link			

Şekil 3: Gazebo Simülatöründe Robot Modelinin Konum (Pose) Verileri

4. TARTIŞMA VE SONUÇ (DISCUSSION)

Bu çalışmada, 3-DoF RRR Planar bir robotun kinematik kapasitesi analitik ve simülatif olarak incelenmiştir. Bulgular bölümünde (Şekil 1) görüldüğü üzere, çalışma alanı beklediği gibi dairesel bir formda ve maksimum 1.2 metre ($L_1+L_2+L_3$) yarıçap sınırlarında hatasız olarak oluşmuştur.

Ters kinematik testlerinde hedeflenen (0.8, 0.2, 0.1) noktası için algoritmanın çözüm üretememesi, sistemin hata yapmasından değil, fiziksel kanunların doğrulanmasından kaynaklanmaktadır. RRR Planar yapılar sadece iki boyutlu düzlemde hareket edecek mafsallara sahip olduğundan, uç işlevcisinin Z ekseninde 0'dan farklı bir noktaya ulaşması sistemin serbestlik derecesi (DoF) mimarisine aykırıdır. Bu kısıt, kodlamadaki kural filtreleriyle başarıyla yakalanmıştır.

Son olarak, Tablo 1'de sunulan hata analizi, Gazebo'da oluşturulan SDF modeli ile Python'da kurulan teorik modelin birebir örtüştüğünü göstermektedir. Denklem (3) kullanılarak yapılan hesaplamada uzamsal hatanın $e = 0.00$ çıkması, uzuv uzunluklarının ve fiziksel birleşim noktalarının (joints) simülasyona kusursuz aktarıldığını kanıtlamaktadır.

KAYNAKÇA

- [1] Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: Modelling, Planning and Control*. Springer Science & Business Media.
- [2] Spong, M. W., Hutchinson, S., & Vidyasagar, M. (2020). *Robot Modeling and Control* (2nd ed.). John Wiley & Sons.

[3] Gazebo Harmonic Documentation, Open Robotics. (2024). Retrieved from <https://gazebo.org/docs>